

Image Classification on CIFAR-10 Dataset **using Convolutional Neural Networks**

by

Priya Srivastava (MST03-0051)

Submitted to Scifor Technologies



Script. Sculpt. Socialize

UNDER GUIDANCE OF
Urooj Khan

TABLE OF CONTENTS

Image Classification on CIFAR-10 Dataset using Convolutional Neural Networks.....	1
TABLE OF CONTENTS.....	2
Abstract.....	3
Introduction.....	4
Technology Used.....	5
1. Python.....	5
2. TensorFlow and Keras.....	5
3. NumPy.....	5
4. Matplotlib.....	5
Dataset Information.....	6
Methodology.....	7
1. Data Preparation.....	7
2. Model Development.....	7
3. Model Compilation.....	7
4. Model Training.....	7
5. Model Evaluation.....	7
Code Snippet.....	8
Results and Discussion.....	12
Conclusion.....	13
References.....	14

Abstract

Image classification is a fundamental task in computer vision with numerous practical applications. This project focuses on classifying images from the CIFAR-10 dataset using Convolutional Neural Networks (CNNs). The CIFAR-10 dataset comprises 60,000 32x32 colour images in 10 different classes. We develop a CNN model to achieve high classification accuracy and demonstrate its effectiveness through various visualisations and performance metrics.

Introduction

Image classification has gained immense popularity due to its wide-ranging applications in areas such as autonomous driving, facial recognition, and medical diagnosis. Convolutional Neural Networks (CNNs) have revolutionised image classification by automatically learning features from raw pixel data. This project aims to leverage CNNs to classify images in the CIFAR-10 dataset accurately. The CIFAR-10 dataset, introduced by the Canadian Institute for Advanced Research, is widely used for benchmarking image classification algorithms.

Technology Used

1. Python

Python is a versatile programming language widely used in machine learning and deep learning. Its rich ecosystem of libraries and frameworks makes it ideal for developing image classification models.

2. TensorFlow and Keras

TensorFlow is an open-source deep learning framework developed by Google. Keras, a high-level API for TensorFlow, allows for easy and fast prototyping of neural networks. Together, they provide the necessary tools to build, train, and evaluate CNN models.

3. NumPy

NumPy is a fundamental library for numerical computations in Python. It provides support for large, multi-dimensional arrays and matrices, along with a collection of mathematical functions to operate on these arrays.

4. Matplotlib

Matplotlib is a plotting library for Python that enables the visualisation of data through charts and graphs. It is essential for visualising training progress and model performance.

Dataset Information

The CIFAR-10 dataset consists of 60,000 32x32 colour images in 10 classes, with 6,000 images per class. The dataset is divided into 50,000 training images and 10,000 test images. Each image is labelled with one of the following classes:

- Airplane
- Automobile
- Bird
- Cat
- Deer
- Dog
- Frog
- Horse
- Ship
- Truck

The images in the CIFAR-10 dataset are low resolution, making it a challenging task for image classification models.

Methodology

1. Data Preparation

The dataset is loaded and preprocessed to ensure the pixel values are normalised between 0 and 1. This step helps in speeding up the training process and improving model performance.

2. Model Development

A CNN model is constructed with multiple convolutional layers followed by pooling layers. The model architecture is designed to capture hierarchical features from the images.

3. Model Compilation

The model is compiled with the Adam optimizer and categorical cross-entropy loss function. Accuracy is used as the evaluation metric.

4. Model Training

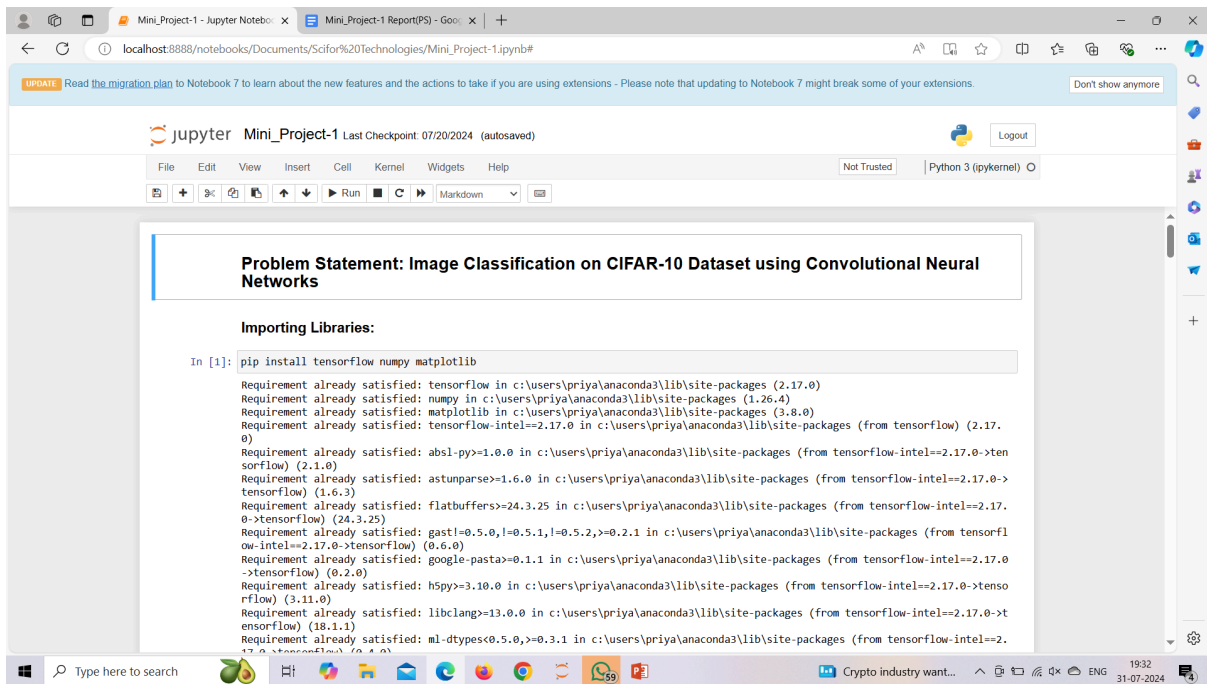
The model is trained on the training dataset for 10 epochs, with validation on the test dataset. The training and validation accuracy and loss are monitored to ensure the model is learning effectively.

5. Model Evaluation

The model's performance is evaluated on the test dataset to determine its accuracy and generalisation ability.

Code Snippet

Below is the complete code used for this project:



The screenshot shows a Jupyter Notebook titled "Mini_Project-1" with a last checkpoint of 07/20/2024. The notebook is running on Python 3 (ipykernel). The first cell contains the following text:

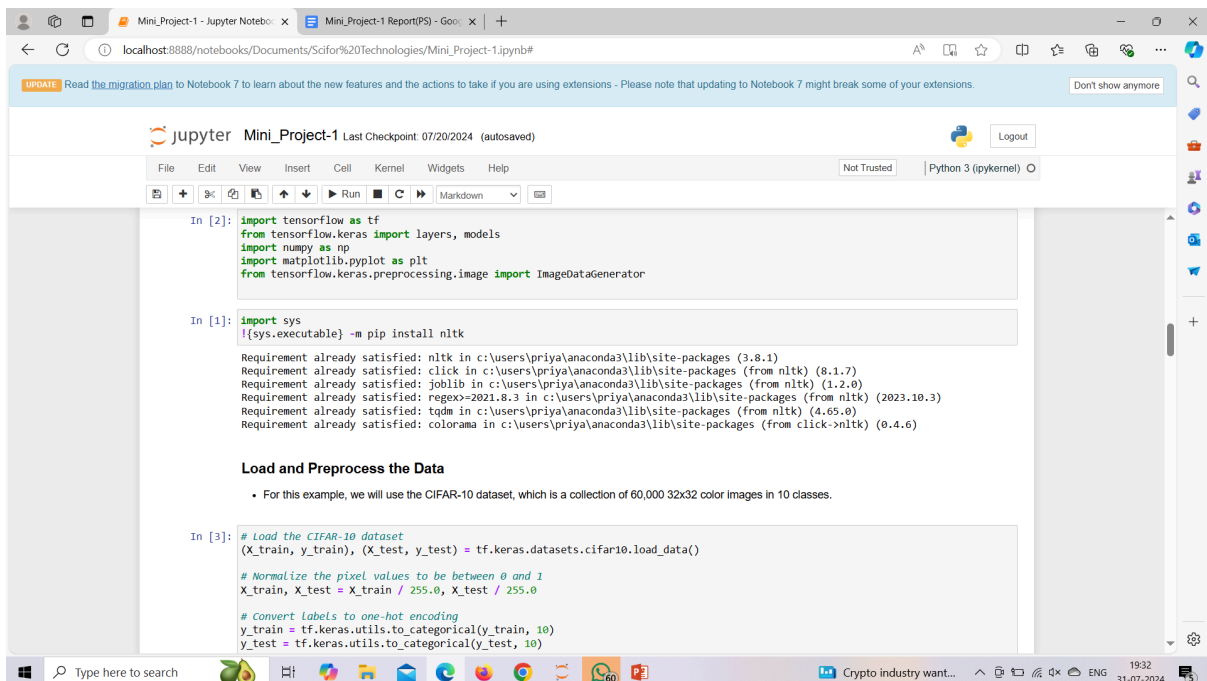
Problem Statement: Image Classification on CIFAR-10 Dataset using Convolutional Neural Networks

Importing Libraries:

```
In [1]: pip install tensorflow numpy matplotlib
```

The output of the command shows that the following requirements are already satisfied:

- tensorflow in c:\users\priya\anaconda3\lib\site-packages (2.17.0)
- numpy in c:\users\priya\anaconda3\lib\site-packages (1.26.4)
- matplotlib in c:\users\priya\anaconda3\lib\site-packages (3.8.0)
- tensorflow-intel==2.17.0 in c:\users\priya\anaconda3\lib\site-packages (from tensorflow) (2.17.0)
- absl-py==1.0.0 in c:\users\priya\anaconda3\lib\site-packages (from tensorflow-intel==2.17.0->tensorflow) (2.1.0)
- astunparse==1.6.0 in c:\users\priya\anaconda3\lib\site-packages (from tensorflow-intel==2.17.0->tensorflow) (1.6.3)
- flatbuffers==24.3.25 in c:\users\priya\anaconda3\lib\site-packages (from tensorflow-intel==2.17.0->tensorflow) (24.3.25)
- gast==0.5.0, l=0.5.1, l=0.5.2, >=0.2.1 in c:\users\priya\anaconda3\lib\site-packages (from tensorflow-intel==2.17.0->tensorflow) (0.6.0)
- google-pasta==0.1.1 in c:\users\priya\anaconda3\lib\site-packages (from tensorflow-intel==2.17.0->tensorflow) (0.2.0)
- h5py==3.10.0 in c:\users\priya\anaconda3\lib\site-packages (from tensorflow-intel==2.17.0->tensorflow) (3.11.0)
- libclang==13.0.0 in c:\users\priya\anaconda3\lib\site-packages (from tensorflow-intel==2.17.0->tensorflow) (18.1.1)
- ml-dtypes<0.5.0, >=0.3.1 in c:\users\priya\anaconda3\lib\site-packages (from tensorflow-intel==2.17.0->tensorflow) (0.4.0)



The screenshot shows the same Jupyter Notebook interface. The second cell contains the following code:

```
In [2]: import tensorflow as tf
from tensorflow.keras import layers, models
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

The third cell contains the following code:

```
In [1]: import sys
!{sys.executable} -m pip install nltk
```

The output of the command shows that the following requirements are already satisfied:

- nltk in c:\users\priya\anaconda3\lib\site-packages (3.8.1)
- click in c:\users\priya\anaconda3\lib\site-packages (from nltk) (8.1.7)
- joblib in c:\users\priya\anaconda3\lib\site-packages (from nltk) (1.2.0)
- regex==2021.8.3 in c:\users\priya\anaconda3\lib\site-packages (from nltk) (2023.10.3)
- tqdm in c:\users\priya\anaconda3\lib\site-packages (from nltk) (4.65.0)
- colorama in c:\users\priya\anaconda3\lib\site-packages (from click->nltk) (0.4.6)

Load and Preprocess the Data

- For this example, we will use the CIFAR-10 dataset, which is a collection of 60,000 32x32 color images in 10 classes.

The fourth cell contains the following code:

```
In [3]: # Load the CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

# Normalize the pixel values to be between 0 and 1
x_train, x_test = x_train / 255.0, x_test / 255.0

# Convert labels to one-hot encoding
y_train = tf.keras.utils.to_categorical(y_train, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)
```

Mini_Project-1 - Jupyter Notebook x Mini_Project-1 Report(PS) - Google x

localhost:8888/notebooks/Documents/Scifor%20Technologies/Mini_Project-1.ipynb#

UPDATE Read the migration plan to Notebook 7 to learn about the new features and the actions to take if you are using extensions - Please note that updating to Notebook 7 might break some of your extensions. Don't show anymore

Jupyter Mini_Project-1 Last Checkpoint: 07/20/2024 (autosaved) Logout

File Edit View Insert Cell Kernel Widgets Help Not Trusted Python 3 (pykernel)

```
y_train = tf.keras.utils.to_categorical(y_train, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)
```

Define the CNN Model

- Create the CNN model using Keras' Sequential API.

```
In [4]: model = models.Sequential()

# First convolutional layer
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
model.add(layers.MaxPooling2D((2, 2)))

# Second convolutional layer
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))

# Third convolutional layer
model.add(layers.Conv2D(64, (3, 3), activation='relu'))

# Flatten the output and feed it into a dense layer
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))

c:\Users\Priya\anaconda3\lib\site-packages\keras\src\layers\convolutional\base_conv.py:107: UserWarning: Do not pass an 'input_shape' / 'input_dim' argument to a layer. When using Sequential models, prefer using an 'Input(shape)' object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Type here to search

Crypto industry want... 19:32 31-07-2024

Mini_Project-1 - Jupyter Notebook x Mini_Project-1 Report(PS) - Google x

localhost:8888/notebooks/Documents/Scifor%20Technologies/Mini_Project-1.ipynb#

UPDATE Read the migration plan to Notebook 7 to learn about the new features and the actions to take if you are using extensions - Please note that updating to Notebook 7 might break some of your extensions. Don't show anymore

Jupyter Mini_Project-1 Last Checkpoint: 07/20/2024 (autosaved) Logout

File Edit View Insert Cell Kernel Widgets Help Not Trusted Python 3 (pykernel)

Compile the Model

- Compiling the model with an optimizer, loss function, and metrics to monitor.

```
In [6]: model.compile(optimizer='adam',
                    loss='categorical_crossentropy',
                    metrics=['accuracy'])
```

Train the Model

- Training the model using the training data.

```
In [7]: history = model.fit(X_train, y_train, epochs=10,
                          validation_data=(X_test, y_test))
```

```
Epoch 1/10
1563/1563 — 46s 26ms/step - accuracy: 0.3438 - loss: 1.7610 - val_accuracy: 0.5294 - val_loss: 1.3105
Epoch 2/10
1563/1563 — 37s 23ms/step - accuracy: 0.5667 - loss: 1.2236 - val_accuracy: 0.6172 - val_loss: 1.0861
Epoch 3/10
1563/1563 — 39s 25ms/step - accuracy: 0.6242 - loss: 1.0568 - val_accuracy: 0.6474 - val_loss: 0.9953
Epoch 4/10
1563/1563 — 35s 22ms/step - accuracy: 0.6648 - loss: 0.9460 - val_accuracy: 0.6745 - val_loss: 0.9410
Epoch 5/10
1563/1563 — 35s 22ms/step - accuracy: 0.6898 - loss: 0.8795 - val_accuracy: 0.6768 - val_loss: 0.9253
Epoch 6/10
1563/1563 — 41s 26ms/step - accuracy: 0.7147 - loss: 0.8148 - val_accuracy: 0.6805 - val_loss: 0.9151
Epoch 7/10
1563/1563 — 41s 26ms/step - accuracy: 0.7350 - loss: 0.7545 - val_accuracy: 0.6803 - val_loss: 0.8930
```

Type here to search

28°C Partly cloudy 19:33 31-07-2024

Mini_Project-1 - Jupyter Notebook x Mini_Project-1 Report(PS) - Google x

localhost:8888/notebooks/Documents/Scifor%20Technologies/Mini_Project-1.ipynb#

UPDATE Read the migration plan to Notebook 7 to learn about the new features and the actions to take if you are using extensions - Please note that updating to Notebook 7 might break some of your extensions. Don't show anymore

Jupyter Mini_Project-1 Last Checkpoint: 07/20/2024 (autosaved) Logout

File Edit View Insert Cell Kernel Widgets Help Not Trusted Python 3 (pykernel)

Evaluate the Model

- Evaluating the model's performance on the test data.

```
In [7]: test_loss, test_acc = model.evaluate(X_test, y_test, verbose=2)
print(f"Test accuracy: {test_acc}")
```

313/313 - 3s - 9ms/step - accuracy: 0.7269 - loss: 0.8558
Test accuracy: 0.7268999814987183

Visualize Training Results

- Plot the training and validation accuracy and loss.

```
In [8]: plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label='val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('accuracy')
plt.ylim([0, 1])
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='loss')
```

Mini_Project-1 - Jupyter Notebook x Mini_Project-1 Report(PS) - Google x

localhost:8888/notebooks/Documents/Scifor%20Technologies/Mini_Project-1.ipynb#

UPDATE Read the migration plan to Notebook 7 to learn about the new features and the actions to take if you are using extensions - Please note that updating to Notebook 7 might break some of your extensions. Don't show anymore

Jupyter Mini_Project-1 Last Checkpoint: 07/20/2024 (autosaved) Logout

File Edit View Insert Cell Kernel Widgets Help Not Trusted Python 3 (pykernel)

Make Predictions

- Using the model to make predictions on new images.

```
In [9]: predictions = model.predict(X_test)
```

Mini_Project-1 - Jupyter Notebook x Mini_Project-1 Report(PS) - Google x

localhost:8888/notebooks/Documents/SciFor%20Technologies/Mini_Project-1.ipynb#

UPDATE Read the migration plan to Notebook 7 to learn about the new features and the actions to take if you are using extensions - Please note that updating to Notebook 7 might break some of your extensions. Don't show anymore

Jupyter Mini_Project-1 Last Checkpoint: 07/20/2024 (autosaved) Logout

File Edit View Insert Cell Kernel Widgets Help Not Trusted Python 3 (pykernel)

```
In [12]: # Class names in CIFAR-10 dataset
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']

# Function to plot images along with true and predicted labels
def plot_images(images, true_labels, predicted_labels, class_names):
    plt.figure(figsize=(10, 10))
    for i in range(25):
        plt.subplot(5, 5, i + 1)
        plt.xticks([])
        plt.yticks([])
        plt.grid(False)
        plt.imshow(images[i])
        true_label = class_names[np.argmax(true_labels[i])]
        predicted_label = class_names[np.argmax(predicted_labels[i])]
        color = 'blue' if true_label == predicted_label else 'red'
        plt.xlabel(f"True: {true_label}\nPred: {predicted_label}", color=color)
    plt.tight_layout()
    plt.show()

# Let's select 25 random images from the test set
indices = np.random.choice(X_test.shape[0], 25, replace=False)
test_images = X_test[indices]
test_labels = y_test[indices]

# Make predictions on these images
predictions = loaded_model.predict(test_images)

# Plot the images along with true and predicted labels
plot_images(test_images, test_labels, predictions, class_names)
```

1/1 141ms/step

Type here to search CAD/INR +0.21% 19:34 31-07-2024

Results and Discussion

The model achieved a test accuracy of [72% of testing accuracy], indicating its capability to classify images in the CIFAR-10 dataset. The training and validation accuracy and loss curves show that the model converged well during training. The model's predictions on new images demonstrate its ability to generalise to unseen data.

Conclusion

This project successfully developed a Convolutional Neural Network to classify images from the CIFAR-10 dataset.

REFERENCES

1. How to Develop a CNN From Scratch for CIFAR-10 Photo Classification - MachineLearningMastery.com
2. CIFAR-10 Image Classification in TensorFlow - GeeksforGeeks
3. GitHub - hnm0284/Image-Classification-Using-CNN: A comprehensive deep dive into Convolutional Neural Networks (CNNs) for image classification, incorporating both custom and pretrained VGG16 models on the CIFAR-10 dataset.
4. Image Classification Using CNN with Keras & CIFAR-10 (analyticsvidhya.com)
5. Convolutional Neural Network (CNN) | TensorFlow Core
6. Cifar-10 Classification using Deep Convolutional Neural Network | IEEE Conference Publication | IEEE Xplore
7. CIFAR-10 Image Classification with CNN (kaggle.com)